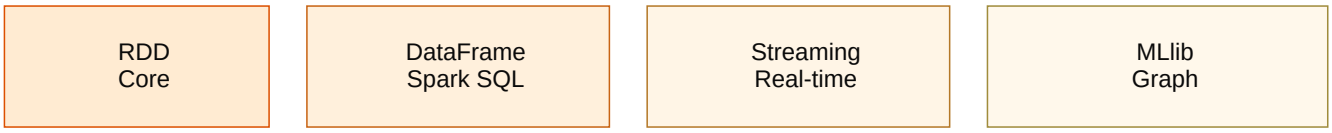
Apache Spark
Concept Revision Notes

Theory Only | No Code | Architecture Diagrams
Basic to Medium | Quick Revision Before Interview



15 Topics | 8 Architecture Diagrams | Comparison Tables | Key Interview Points

Table of Contents

1. What is Spark? Why Spark?
2. Spark Architecture (Diagram)
3. Spark Execution Model (Diagram)
4. RDD Concepts
5. DataFrame & Dataset
6. Transformations vs Actions (Diagram)
7. Partitions & Parallelism (Diagram)
8. Joins in Spark (Diagram)
9. Window Functions
10. Caching & Persistence
11. Shuffle & Performance (Diagram)
12. Spark Memory Model
13. Optimization Techniques
14. Structured Streaming (Diagram)
15. File Formats Comparison

Config Cheat Sheet

20 Key Interview Points

1. What is Apache Spark?

Apache Spark is a unified analytics engine for large-scale data processing. It provides in-memory computing capabilities that make it up to 100x faster than Hadoop MapReduce for iterative workloads.

Why Spark Replaced MapReduce

- In-memory processing: intermediate results stay in RAM, not written to disk between stages
- Unified engine: batch + streaming + ML + graph in one framework
- Lazy evaluation: builds execution plan first, optimizes before running
- Interactive: supports notebooks and REPL for exploratory analysis
- Easy APIs: DataFrame/SQL much simpler than writing raw MapReduce Java

Spark vs MapReduce

Aspect	MapReduce	Apache Spark
Speed	Slow (disk I/O between stages)	100x faster (in-memory)
Processing	Batch only	Batch + Stream + Interactive
Ease of Use	Complex Java API	Simple Python/SQL/Scala API
Fault Tolerance	Replication (3x storage)	Lineage (recompute on failure)
Real-time	Not supported	Structured Streaming
Iterative (ML)	Very slow (rewrite to disk)	Fast (cache in memory)
Cost	Cheap (disk)	More RAM needed

Spark Ecosystem

- Spark Core: RDD, task scheduling, memory management, fault recovery
- Spark SQL: DataFrames, Datasets, Catalyst optimizer, Hive integration
- Spark Streaming: micro-batch (Structured Streaming) and continuous processing
- MLlib: distributed machine learning (classification, regression, clustering)
- GraphX: graph processing (PageRank, connected components)

KEY: In interviews, always say "DataFrame API with Spark SQL" is the modern approach. RDD is legacy.

2. Spark Architecture

DIAGRAM 1: Spark Cluster Architecture

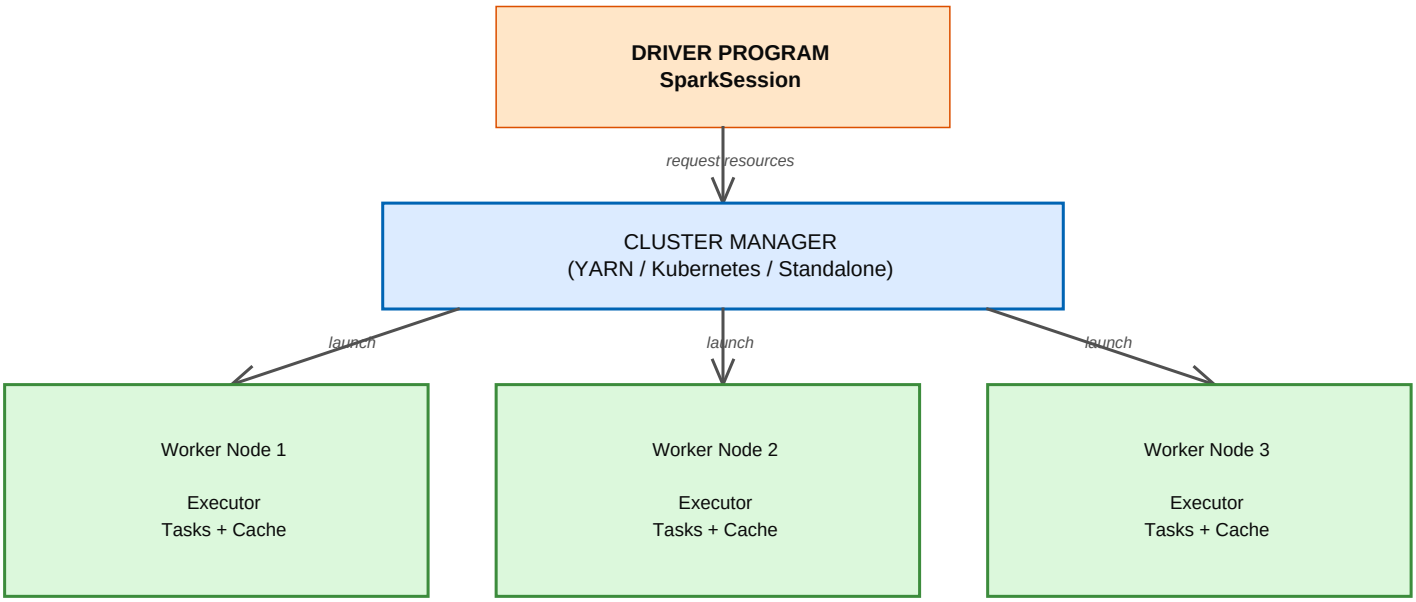


Fig 1: Driver sends tasks to Executors via Cluster Manager

Key Components

- SparkSession: single entry point (replaces SparkContext in Spark 2.0+)
- Driver: runs your main program, creates DAG, distributes tasks to executors
- Cluster Manager: allocates resources. Types: YARN (most common), Kubernetes, Standalone, Mesos
- Worker Node: physical/virtual machine in the cluster
- Executor: JVM process on a worker. Runs tasks and holds cached data. One or more per worker.
- Task: smallest unit of work. One task processes one partition of data.

DIAGRAM 2: Job --> Stage --> Task Hierarchy

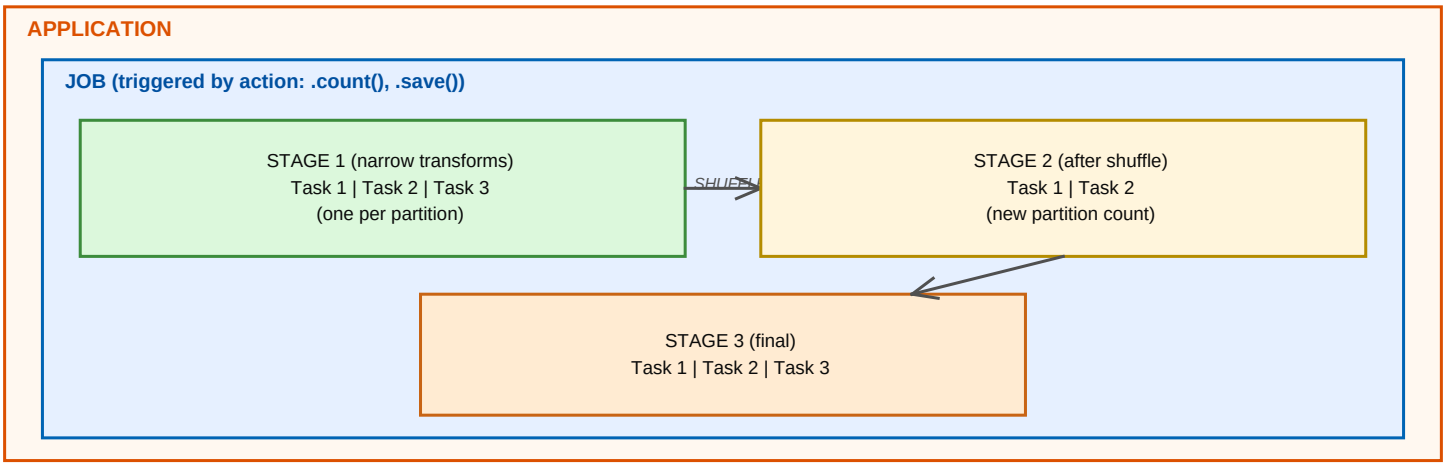


Fig 2: Each action creates a Job. Shuffle boundaries create Stage breaks. Each partition = 1 Task.

KEY: Interview question: "What triggers a new stage?" Answer: any wide transformation (shuffle) like groupBy, join, repartition, distinct, orderBy.

3. Spark Execution Model

DIAGRAM 3: From User Code to Execution

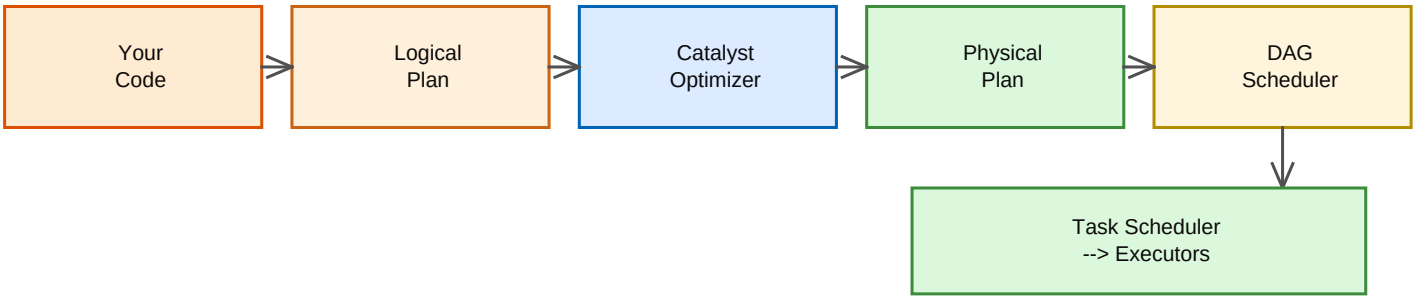


Fig 3: Catalyst Optimizer rewrites your logical plan for best performance

Lazy Evaluation

- Transformations (map, filter, join, groupBy) are NOT executed immediately
- Spark builds a DAG (Directed Acyclic Graph) of all transformations
- Only when an ACTION is called (count, show, save) does execution begin
- Benefit: Catalyst can optimize the entire plan before running anything
- Example: if you filter THEN join, Catalyst may push filter before join (predicate pushdown)

Catalyst Optimizer - What It Does

- Predicate pushdown: moves WHERE filters closer to data source (read less data)
- Column pruning: only reads columns actually used in the query
- Join reordering: puts smaller table on build-side of hash join
- Constant folding: pre-computes constant expressions at compile time
- Combines multiple aggregations into single pass

KEY: Catalyst only works with DataFrame/SQL API, NOT with RDD. This is why RDD is slower.

4. RDD Concepts

RDD (Resilient Distributed Dataset) is the original Spark abstraction. It is a read-only, partitioned collection of records distributed across the cluster.

5 Key Properties of RDD

- Partitioned: data split across multiple nodes for parallel processing
- Immutable: once created, cannot be changed. Transformations create new RDDs.
- Resilient: if a partition is lost, Spark replays the lineage (sequence of transformations) to recompute it
- Lazy: transformations are recorded but not executed until an action is called
- Typed: RDD[String], RDD[Int] -- type checked at compile time (Scala/Java)

RDD Lineage (Fault Tolerance)

Each RDD remembers HOW it was created (which parent RDD + which transformation). If a partition is lost (node failure), Spark traces back the lineage and recomputes only the lost partition. This is cheaper than replication (Hadoop stores 3 copies of every block).

When to Use RDD vs DataFrame

Aspect	RDD	DataFrame
Optimization	No Catalyst optimizer	Full Catalyst + Tungsten
Performance	Slower (no optimization)	2-10x faster
API	Functional (map/filter/reduce)	Declarative (SQL/select/filter)
Schema	No schema	Full schema (named columns)
Use Case	Low-level control, legacy code	All modern Spark work

KEY: Modern answer: "Always use DataFrame unless you need low-level byte manipulation or custom partitioning that DataFrame cannot

express."

5. DataFrame & Dataset

What is a DataFrame?

- A distributed collection of rows organized into named columns (like a table)
- Has a schema: each column has a name, data type, and nullable flag
- Optimized by Catalyst (query optimizer) and Tungsten (memory/CPU optimizer)
- Available in Python, Scala, Java, R. Python API is most popular.
- Immutable: operations return NEW DataFrames, never modify the original

DataFrame vs Dataset vs RDD

Feature	RDD	DataFrame	Dataset
Available In	Python/Scala/Java	Python/Scala/Java/R	Scala/Java only
Schema	No	Yes (named columns)	Yes (typed)
Catalyst Optimizer	No	Yes	Yes
Type Safety	Compile-time	Runtime only	Compile-time
Serialization	Java/Kryo	Tungsten (binary)	Tungsten (binary)
Performance	Slowest	Fast	Fast
When to Use	Never (legacy)	Python users (always)	Scala with type safety

Schema Concepts

- StructType: container for the full schema (list of columns)
- StructField: one column definition (name, dataType, nullable)
- Common types: StringType, IntegerType, LongType, DoubleType, BooleanType, DateType, TimestampType, ArrayType, MapType
- Schema can be inferred (from data) or explicitly defined (safer for production)

KEY: Always define schema explicitly in production. Schema inference reads entire file first, which is slow and unreliable for large datasets.

6. Transformations vs Actions

DIAGRAM 4: Narrow vs Wide Transformations

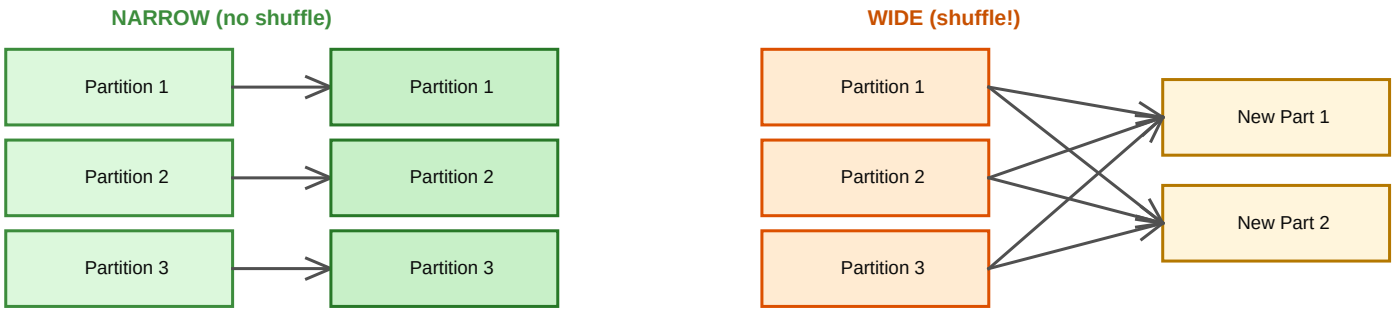


Fig 4: Narrow = 1-to-1 partition mapping. Wide = all-to-all shuffle (expensive).

Narrow Transformations (no data movement)

- select, filter/where, withColumn, withColumnRenamed, drop, map, flatMap, union
- Each partition is processed independently -- no network I/O
- Multiple narrow transforms fuse into a single stage (pipelined)

Wide Transformations (shuffle required)

- groupBy, reduceByKey, aggregateByKey, join, distinct, repartition, orderBy/sort
- Data must be redistributed across partitions via network
- Creates a STAGE BOUNDARY in the execution plan
- Shuffle = most expensive operation in Spark (network + disk I/O)

Actions (trigger execution)

- collect() - bring all data to driver (DANGER: OOM if data is large)
- count() - count rows
- show() - display first N rows
- first() / take(N) - get N rows
- save / write - write to storage
- foreach() - apply function to each row
- reduce() - aggregate to single value

7. Partitions & Parallelism

A partition is the basic unit of parallelism in Spark. Each partition is processed by exactly one task on one executor core.

Key Partition Rules

- More partitions = more parallelism (up to available cores)
- Too few partitions = underutilized cluster (cores sitting idle)
- Too many partitions = excessive scheduling overhead + small files problem
- Rule of thumb: 2-4 partitions per CPU core
- Target partition size: 128MB - 256MB each
- Default shuffle partitions: 200 (spark.sql.shuffle.partitions) -- often needs tuning!

DIAGRAM 5: Repartition vs Coalesce

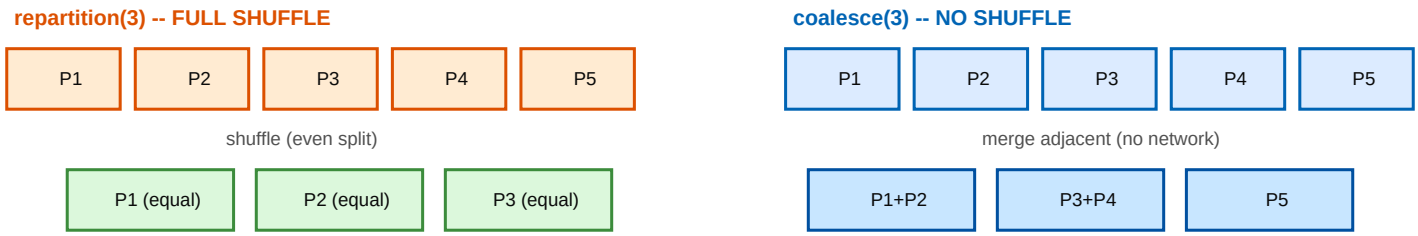


Fig 5: repartition = even but costly shuffle. coalesce = free but potentially uneven.

When to Use Which

- coalesce(N): reduce partitions before writing (avoid small output files). No shuffle = fast.
- repartition(N): increase partitions OR need even distribution (before groupBy/join). Full shuffle.
- repartition(N, "col"): repartition by column value (co-locate same keys for joins)

8. Joins in Spark

DIAGRAM 6: Join Types

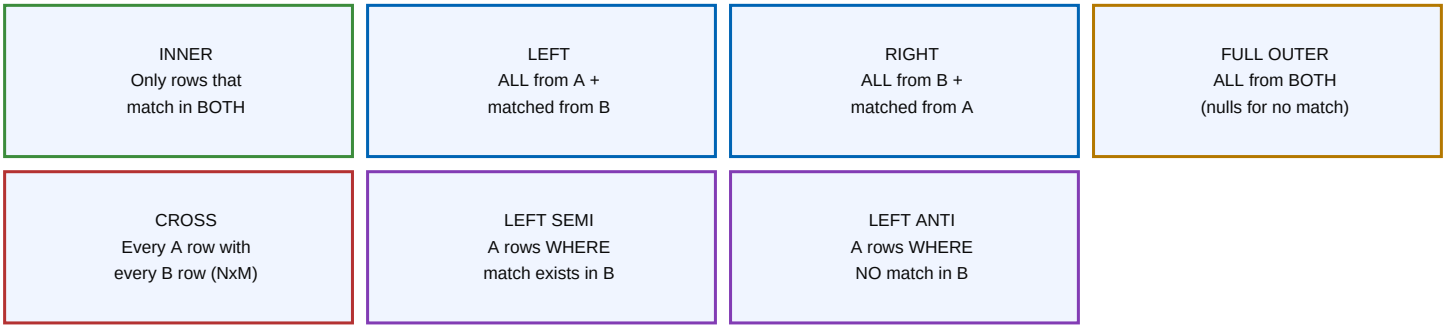


Fig 6: 7 join types. Inner is default. Left Anti is "not in" equivalent.

Join Strategies (How Spark Executes Joins)

- Broadcast Hash Join: small table (< 10MB) broadcast to all executors. Fastest. No shuffle.
- Sort Merge Join: both tables sorted by join key, then merged. Default for large-large joins.
- Shuffle Hash Join: both tables shuffled by key, hash table built. Good for medium tables.

KEY: If one table is < 10MB, Spark auto-broadcasts it. Override threshold: spark.sql.autoBroadcastJoinThreshold (default 10MB). Set to -1 to disable.

9. Window Functions

Window functions compute a value for each row based on a "window" of related rows. Unlike GROUP BY, window functions do NOT collapse rows -- every row keeps its original columns.

Window Function Categories

- Ranking: ROW_NUMBER (unique rank), RANK (ties get same rank, skips), DENSE_RANK (ties, no skip), NTILE (buckets)
- Value: LAG (previous row), LEAD (next row), FIRST_VALUE, LAST_VALUE
- Aggregate: SUM, AVG, COUNT, MIN, MAX over the window (running totals etc.)

Window Specification

- PARTITION BY: defines groups (like GROUP BY but keeps all rows)
- ORDER BY: defines row order within each partition
- ROWS BETWEEN: defines frame (which rows to include in calculation)
  - Common frames: UNBOUNDED PRECEDING to CURRENT ROW (running total), 6 PRECEDING to CURRENT ROW (7-day rolling)

KEY: ROW_NUMBER + PARTITION BY is the #1 most asked Spark/SQL interview pattern. "Get top N per group" = ROW_NUMBER OVER (PARTITION BY group ORDER BY value DESC) then filter rn <= N.

10. Caching & Persistence

Storage Levels

Level	Where Stored	Serialized?	Best For
MEMORY_ONLY	RAM only	No	Small DataFrames used repeatedly
MEMORY_AND_DISK	RAM + spill to disk	No	Large DataFrames, cannot fit in RAM
DISK_ONLY	Disk only	Yes	Very large, infrequent access
MEMORY_ONLY_SER	RAM, serialized	Yes	Memory pressure, less GC overhead
OFF_HEAP	Off-heap memory	Yes	Avoid GC pauses

When to Cache vs Not

- CACHE when: DataFrame is reused 2+ times, iterative ML algorithms, joining same table repeatedly
- DO NOT cache when: DataFrame used only once, very large (>50% of cluster RAM), streaming
- cache() = persist(MEMORY_ONLY). Use persist() for other storage levels.
- unpersist() to free memory when done
- Checkpoint: saves to reliable storage (HDFS/S3), breaks lineage. Use for very long chains.

11. Shuffle & Performance

DIAGRAM 7: Shuffle Process

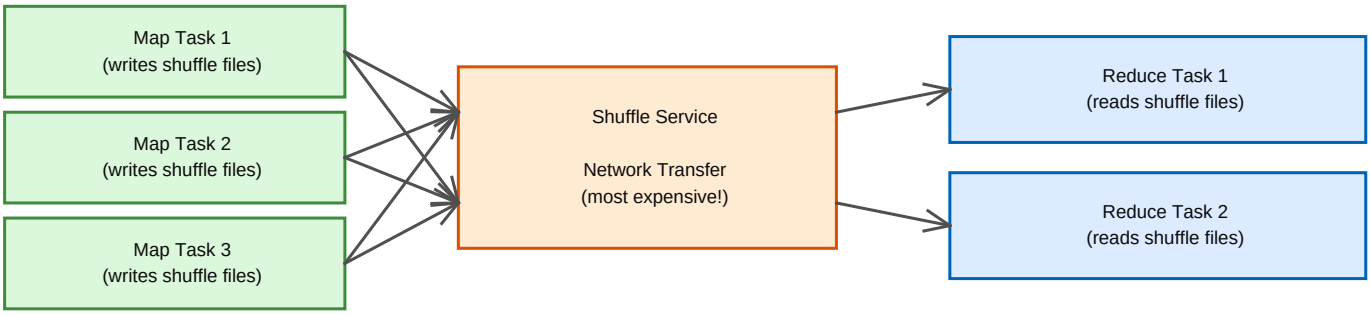


Fig 7: Shuffle = map tasks write to local disk, reduce tasks read over network.

Why Shuffle is Expensive

- Disk I/O: map tasks write shuffle files to local disk
- Network I/O: reduce tasks pull shuffle data from ALL map tasks over network

- Serialization: data serialized/deserialized during transfer
- Spill: if executor memory is full during shuffle, data spills to disk (even slower)

How to Reduce Shuffle

- Filter early: reduce data volume before groupBy/join
- Broadcast small tables: avoid shuffle entirely for small-large joins
- Use coalesce instead of repartition when reducing partitions
- Pre-aggregate before join: reduce rows before the expensive shuffle
- Bucketing: pre-shuffle data on write so future joins skip shuffle

12. Spark Memory Model

Each executor divides its heap memory into regions. Understanding this helps tune for OOM errors.

Executor Memory Layout

- Total Executor Memory: set by spark.executor.memory (e.g., 4g)
- Reserved Memory: 300MB fixed (Spark internal)
- User Memory (40%): your UDFs, data structures, metadata
- Spark Memory (60%): split between Execution and Storage
- - Execution Memory: shuffle, joins, sorts, aggregations
- - Storage Memory: cached DataFrames and broadcast variables
- Execution and Storage can borrow from each other (Unified Memory Manager)

KEY: OOM on executor? Options: (1) Increase spark.executor.memory, (2) Increase partitions (smaller data per task), (3) Reduce broadcast table size, (4) Spill to disk with MEMORY_AND_DISK.

13. Optimization Techniques

Query Optimization (Catalyst)

- Predicate pushdown: push WHERE filters to data source (Parquet only reads matching row groups)
- Column pruning: only read columns used in query (huge savings on wide tables)
- Join reordering: place smaller table on build-side
- Constant folding: pre-compute constant expressions

Data-Level Optimization

- Use columnar formats: Parquet or ORC (not CSV/JSON for analytics)
- Partition data by common filter column (date, region)
- Z-ORDER / clustering: sort data by frequently filtered columns within partitions
- Compact small files: coalesce or OPTIMIZE (Delta Lake)

Runtime Optimization

- Adaptive Query Execution (AQE): Spark 3.0+ runtime optimization
- - Auto-coalesce shuffle partitions (combines tiny partitions)
- - Auto-broadcast small tables (detected at runtime, not just plan time)
- - Skew join optimization (splits skewed partitions automatically)
- Enable AQE: spark.sql.adaptive.enabled = true (default true in Spark 3.2+)

Common Performance Problems

Problem	Symptom	Fix
Data Skew	1 task takes 10x longer	Salt key, repartition, AQE skew join
Too Many Small Files	Slow reads, S3 throttling	coalesce before write, OPTIMIZE
Shuffle Spill	Spill metrics > 0 in Spark UI	More memory or more partitions
Full Table Scan	Query scans all files	Partition pruning, predicate pushdown
OOM Error	Executor killed by YARN	More memory, fewer partitions, no collect()
Driver OOM	Driver process crashes	Avoid collect() on large data

14. Structured Streaming Concepts

DIAGRAM 8: Structured Streaming Model

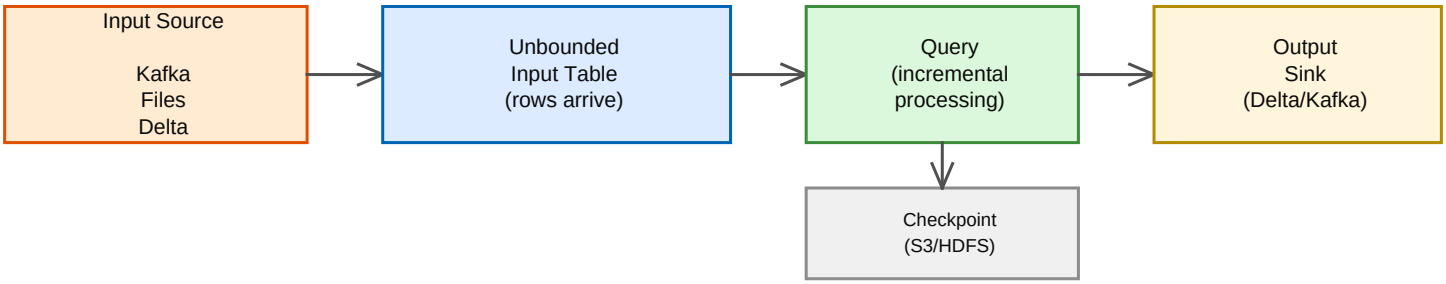


Fig 8: Streaming treats input as an ever-growing table. Processes incrementally.

Key Streaming Concepts

- Micro-batch (default): processes data in small batches (e.g., every 10 seconds)
- Continuous processing: experimental, true record-by-record (~1ms latency)
- Trigger types: processingTime("10 seconds"), once (one batch then stop), availableNow (process all available then stop)
- Watermark: tells Spark how long to wait for late data before closing a window
- Checkpoint: saves processing progress to fault-tolerant storage. Enables exactly-once.
- Output modes: append (new rows only), complete (full result table), update (changed rows only)

15. File Formats Comparison

Format	Type	Compression	Schema	Splittable	Best For
CSV	Row	None/Gzip	No (inferred)	Yes (uncompressed)	Human readable, small data
JSON	Row	None/Gzip	Semi (inferred)	Yes (line-delimited)	API responses, logs
Parquet	Columnar	Snappy/Gzip	Yes (embedded)	Yes	Analytics, DWH (best default)
ORC	Columnar	Zlib/Snappy	Yes (embedded)	Yes	Hive ecosystem
Avro	Row	Snappy/Deflate	Yes (separate)	Yes	Kafka messages, schema evolution
Delta	Columnar+	Snappy	Yes (log)	Yes	Lakehouse (ACID, time travel)

KEY: Default choice for analytics: Parquet. For lakehouse: Delta Lake. For Kafka: Avro. Never use CSV/JSON for large-scale analytics.

Spark Configuration Cheat Sheet

Config	Default	Recommended	What It Does
spark.executor.memory	1g	4g-16g	RAM per executor
spark.executor.cores	1	4-5	CPU cores per executor
spark.driver.memory	1g	2g-8g	RAM for driver program
spark.sql.shuffle.partitions	200	Auto (AQE)	Partitions after shuffle
spark.sql.autoBroadcastJoinThreshold	10MB	10-100MB	Auto-broadcast small tables
spark.sql.adaptive.enabled	true	true	Adaptive Query Execution
spark.serializer	Java	Kryo	Faster serialization
spark.speculation	false	true	Re-run slow tasks on idle nodes
spark.default.parallelism	Total cores	2-4x cores	Default RDD partitions
spark.sql.files.maxPartitionBytes	128MB	128-256MB	Max bytes per file partition

20 Key Interview Points to Memorize

1. Spark is 100x faster than MapReduce because it processes data in-memory.
2. Always use DataFrame API (not RDD) for Catalyst optimizer benefits.
3. Lazy evaluation: nothing executes until an action (count, show, save) is called.
4. An action triggers a Job. A shuffle creates a Stage boundary. Each partition = 1 Task.
5. Narrow transforms (map, filter) do NOT shuffle. Wide transforms (groupBy, join) DO shuffle.
6. Shuffle is the most expensive operation -- minimize it by filtering early and broadcasting small tables.
7. repartition = full shuffle (even distribution). coalesce = no shuffle (just merge, may be uneven).
8. Target partition size: 128-256MB. Target partition count: 2-4x total cores.
9. Broadcast Hash Join is fastest (small table copied to all executors, no shuffle).
10. AQE (Adaptive Query Execution) auto-optimizes at runtime. Enable in Spark 3.0+.
11. cache() = persist(MEMORY_ONLY). Only cache DataFrames reused 2+ times.
12. Parquet is the best file format for analytics (columnar, compressed, predicate pushdown).
13. Predicate pushdown: WHERE filters pushed to data source to read less data.
14. Column pruning: only read columns used in the query (huge savings on wide Parquet tables).
15. OOM? Increase executor memory, increase partitions, avoid collect() on large data.
16. Data skew: one partition much larger. Fix: salt the key or enable AQE skew join.
17. Structured Streaming treats input as an unbounded table, processes incrementally.
18. Watermark = how long to wait for late data in streaming windows.
19. SparkSession is the single entry point. SparkContext is legacy (pre-2.0).
20. EXPLAIN shows the query plan. Use it to verify predicate pushdown and join strategy.